

Fine-tuning bilingual embeddings for Emotion mining in Hindi

A Dissertation Submitted to the University of Hyderabad
in Partial fulfillment of the Degree of

Master of Technology
in
ARTIFICIAL INTELLIGENCE

by
Garima Jain
18MCM14



SCHOOL OF COMPUTER AND INFORMATION SCIENCES

UNIVERSITY OF HYDERABAD

(P.O.) Central University, Gachibowli,

Hyderabad – 500 046

Telangana, India

June 2020



CERTIFICATE

This is to certify that the dissertation entitled, "**Fine-tuning bilingual embeddings for Emotion mining in Hindi**" submitted by **Garima Jain**, bearing Regd. No. **18MCM14**, in partial fulfillment of the requirements for the award of **Master of Technology in Artificial Intelligence** is a bonafide work carried out by her under my supervision and guidance.

The dissertation has not been submitted previously in part or in full to this or any other University or Institution for the award of any degree or diploma.

Signature of the Supervisor

Prof. Kavi Narayana Murthy

School of Computer and Information Sciences

University of Hyderabad

Signature of the Dean

Prof. Chakravarthy Bhagwati

School of Computer and Information Sciences

University of Hyderabad

DECLARATION

I, **Garima Jain**, hereby declare that this dissertation titled, "**Fine-tuning bilingual embeddings for Emotion mining in Hindi**" submitted by me under the guidance and supervision of **Prof. Kavi Narayana Murthy** is a bonafide research work. I also declare that it has not been submitted previously in part or in full to this University or other University or Institution for the award of any degree or diploma.



Signature of the Student

Date: June 24, 2020

Name: Garima Jain

Regd. No.: 18MCM14

ACKNOWLEDGEMENT

I am extremely thankful to my supervisor Professor Kavi Narayana Murthy for the motivation and tireless efforts to help me to get deep knowledge of the research area and supporting me throughout the life cycle of my M. Tech. dissertation work. He always encouraged to go into the fine details of concepts, and, question everything for a better understanding, which definitely helped to study from a different perspective. He provided the freedom to choose our own path by exploring different areas of NLP, and, do what interests us the most, and always guided through the journey. Especially, the extensive comments, healthy discussions, and fruitful interactions with him had a direct impact on the final form and quality of M. Tech. dissertation work.

I am also thankful to Prof. Chakravarthy Bhagwati, Dean, School of Computer and Information Sciences(SCIS), for his fruitful guidance through the early period of chaos and confusions. I wish to thank the faculty members and supporting staff of SCIS and Natural Language Engineering(NLE) Lab, for providing the required resources and infrastructure to work on the thesis and their heartiest co-operation. I extend my gratitude to Indira Gandhi Memorial Library(IGML), University of Hyderabad, for allowing access to a rich collection of books and online journals and digital libraries.

This thesis would not have been possible without the continuous support and feedback from NLE Lab members : Renjith Ravindran, Ram Anirudh Cherukupally, Shivam Soni and Manan Singh. Long discussions and interactive sessions with them gave an unparalleled opportunity to resolve all the doubts, work on new ideas and gather new knowledge.

My deepest regards to my friends and parents for their blessings, affection and encouragement.

(Garima Jain)

ABSTRACT

Emotion mining in text is the task of identifying emotion conveyed in short texts like reviews, opinions, news headlines, etc. Although many works exist in English language for emotion mining, as mentioned in [Yadollahi et al., 2017], we found only one work for emotion mining in Hindi [Ahmad et al., 2020]. In this work, we investigate the task of Emotion mining in Hindi, by performing cross-lingual transfer, from resource rich language(English) to resource poor language(Hindi), using a simple LSTM architecture . The proposed architecture in [Ahmad et al., 2020] is quite complex and even after transferring knowledge from rich English resources using cross-lingual embeddings and transfer learning techniques, results are not quite good. We obtain significantly better results, using a simple model architecture with fewer parameters to be learnt.

We also explore the fine-tuning of cross-lingual embeddings for emotion mining in Hindi, which, to the best of our knowledge, has not been explored yet. Task-specific embeddings capture task related information, besides, semantic information. The results show a little improvement, over the baseline and cross-lingual models, which leaves the room for exploration of better approaches to fine tune embeddings, specific to a task, for resource poor languages.

Contents

CERTIFICATE	i
DECLARATION	ii
ACKNOWLEDGEMENT	iii
ABSTRACT	iv
List of Figures	viii
List of Tables	ix
List of Abbreviations	x
1 Introduction	1
1.1 Motivation	1
1.2 Contribution	3
1.3 Outline	3
2 Background	5
2.1 Emotion mining works	5
2.2 Learning bilingual embeddings	6
2.3 Learning task-specific embeddings	7
3 Machine Learning	9
3.1 Optimisation Strategies	9
3.1.1 Gradient Descent	9
3.1.2 Momentum based Gradient Descent	10
3.1.3 Frequency of Update of Parameters	10
3.2 Non-linear Activation functions	11
3.2.1 Logistic	12
3.2.2 Tanh	12

3.2.3	Rectified Linear Unit(ReLU)	12
3.2.4	Leaky ReLU	12
3.3	Loss Functions	13
3.3.1	Mean Squared Error(MSE)	13
3.3.2	Cross Entropy	13
3.3.3	Softmax function	14
3.4	Evaluating Models	15
3.4.1	Precision	16
3.4.2	Recall	16
3.4.3	F1 score	16
3.4.4	Averaging the metrics over multiple classes	16
4	Artificial Neural Networks	18
4.1	Inspiration from biological neurons [Jain et al., 1996]	18
4.2	Feed Forward Neural Networks	19
4.3	Sequence Modeling	20
4.3.1	Recurrent Neural Networks(RNN)	20
4.3.2	Long Short Term Memory(LSTM)	22
5	Learning Word Embeddings	24
5.1	Word2Vec	24
5.1.1	Continuous Skip Gram	25
5.1.2	Continuous Bag of Words(CBoW)	25
5.2	FastText	27
6	Proposed Work	28
6.1	Training models for Emotion mining in Hindi	28
6.1.1	Model I : Baseline Model	28
6.1.2	Model II : Cross-lingual transfer	29
6.1.3	Model III : Learning a mapping from monolingual to task-specific embedding space	31
6.2	Reference for comparisons	33
6.3	Description of datasets	34
6.3.1	Hindi emotion dataset	34

6.3.2	English emotion dataset	35
6.4	Experiments, Results and Analysis	36
6.4.1	Experiment I : Hindi baseline	36
6.4.2	Experiment II : Model Pre-training	37
6.4.3	Experiment III : Cross-lingual transfer	38
6.4.4	Experiment IV : Using Hindi task-specific embeddings	39
6.4.5	Experiment V : Bilingual embeddings	40
7	Conclusion & Future Scope	42
7.1	Conclusion	42
7.2	Future Work	43

List of Figures

4.1	Feed Forward Neural Network	19
4.2	Recurrent Neural Network	21
4.3	an LSTM cell	23
5.1	Continuous Skip Gram	26
5.2	Continuous Bag of Words	26
6.1	Model I(Baseline) Architecture	28
6.2	Model II(Cross-lingual) Architecture	29

List of Tables

6.1	Model Settings and Hyperparameters	33
6.2	Baseline Results	34
6.3	Cross-lingual with different transfer learning techniques	34
6.4	Hindi emotion dataset : Distribution of samples across categories . .	35
6.5	English emotion dataset : Distribution of samples across categories .	36
6.6	Model Settings and Hyperparameters	37
6.7	Results I	37
6.8	Results II	38
6.9	Results III	38
6.10	Results IV.A	39
6.11	Results IV.B	40
6.12	Bilingual embedding space	41

List of Abbreviations

HCI	Human Computer Interaction
GD	Gradient Descent
MSE	Mean Squared Error
ML	Machine Learning
TP	True Positive
TN	True Negative
FP	False Positive
FN	False Negative
ReLU	Rectified Linear Unit
ANN	Artificial Neural Network
RNN	Recurrent Neural Network
LSTM	Long Short Term Memory
CBoW	Continuous Bag of Words
OoV	Out of Vocabulary

Chapter 1

Introduction

Emotion mining aims to capture emotions that people express through their facial expressions, writings, speech, movements etc. Emotion mining in text is the task of identifying emotion from short pieces of text like microblogs, chat messages, reviews, news headlines, and so on. According to [Fox, 2008], emotion is defined as *discrete and consistent responses to internal and external events that have a particular significance for the organism and emotions have short-term duration*. The author also gives clear definitions of the terms *feeling*, *mood*, *affect* and distinguishes them from the term "emotion".

1.1 Motivation

Emotion mining is a very popular research area in disciplines like neuroscience, psychology and cognitive sciences and now it has become an interesting area of research in computer science also. Some of its applications include identifying if a customer's complaint email is "emotional"(e.g expresses customer's dissatisfaction) so that they are handled appropriately [Gupta et al., 2013], developing emotion-sensitive Human Computer Interaction(HCI) systems [Voeffray, 2011], investigating the impact of emotions on author profiling to identify author's age and gender [Rangel and Rosso, 2016], building tools to estimate the risk of depression before its onset using behavioral cues from tweets of users [De Choudhury et al., 2013] and many others.

The survey [Yadollahi et al., 2017] puts both Emotion Mining and Opinion Mining under the umbrella of **Sentiment Analysis**. Opinion Mining is the task of extracting user’s opinion(positive, negative, or neutral/conflict) from a piece of writing. Emotion mining is closely related to the task of Opinion Mining and differ from it subtly. While Opinion Mining aims to extract the opinion that a user expresses, Emotion Mining deals with extraction of fine-grained emotion expressed by the user. Where opinions are retained for longer duration, emotions are short-lived. [Yadollahi et al., 2017] give an example differentiating between opinion and emotion : *”My family thinks it’s a good decision to continue my education overseas, though they feel sad to miss me”*, this sentence expresses **positive** opinion, but **sad** emotion.

A number of theories [Ekman, 1992, Plutchik, 1984] have been proposed regarding the different types of emotion felt by humans in different scenarios, most common being *happiness, sadness, anger* etc. The survey [Yadollahi et al., 2017] summarises major works in Emotion mining. The authors discuss various supervised, unsupervised and lexicon-based approaches that researchers have used for mining emotions from text. Popular approaches that have been used are : Rule-based approaches [Neviarouskaya et al., 2007, Chaumartin, 2007, Neviarouskaya et al., 2009], lexicon-based approaches [Kim et al., 2010, Chaumartin, 2007] using Wordnet Affect lexicon [Strapparava et al., 2004], ensemble techniques [Gupta et al., 2013, Bhowmick et al., 2009], Support Vector Machine based classifier [Luyckx et al., 2012] etc. The survey also talks about various emotion-related resources including datasets and lexicons that have been created by many researchers over many years. Emotion lexicons [Strapparava et al., 2004, Mohammad and Turney, 2010b, Mohammad, 2012] impart prior knowledge about the type and strength of emotion carried by different words and phrases.

Almost all of the research in Emotion Mining has been done for English language texts. There has not been much research in Emotion mining, and that too in Hindi language, which remains a resource scarce language. Although techniques used for emotion mining in English can be employed for other languages, other languages don’t have enough resources as there are available for English language. Due to resource scarcity in many languages, cross-lingual approaches have been applied and authors have successfully developed frameworks for performing different NLP

tasks using cross-lingual knowledge, taking relevant information from resource rich languages. So, for resource-poor languages like Hindi, such cross-lingual approaches are required to build high quality emotion detecting systems. [Ahmad et al., 2020] is the only existing work that explores Emotion mining in Hindi text. They employed transfer learning and cross-lingual approaches to take advantage of rich annotated datasets available in English.

There also have been studies which have explored the fine-tuning of general embeddings, specific for the task. These embeddings incorporate task-specific features, along with semantic information. Their effectiveness has been explored in works like [Tang et al., 2014, Agrawal et al., 2018] in which authors show that task-specific embeddings perform better than general embeddings, by mapping same sentiment/emotion words nearby.

Authors in [Sakuma and Yoshinaga, 2019], worked on learning task-specific embeddings for resource-poor languages by applying cross-lingual transfer for the tasks of Sentiment Analysis and Topic Modeling. To the best of our knowledge, there exist no work in Hindi, which have learnt task-specific embeddings using cross-lingual transfer.

1.2 Contribution

In our work, we train a simple LSTM model using the Hindi emotion dataset provided by [Ahmad et al., 2020]. To improve upon this baseline model, we investigate cross-lingual transfer by utilising English emotion annotated datasets. We compare our results with [Ahmad et al., 2020] and show that our simple model gives much better results. We also investigate the fine-tuning of Hindi cross-lingual embeddings, for the task of emotion mining, by learning linear and non-linear mappings using neural network architectures.

1.3 Outline

In Chapter 2, we review the literature related to Emotion mining works and learning of bilingual embeddings. Chapters 3, 4, 5 respectively talk about Machine learning, Artificial Neural Networks and Word Embedding algorithms in detail. Chapter 6

covers the details of proposed work and experiments that we performed and analysis. Finally, in Chapter 7, we conclude the work and leave some open questions that can be taken up as a research topic for anyone interested in this area.

Chapter 2

Background

2.1 Emotion mining works

In the survey [Yadollahi et al., 2017], the authors summarised major emotion mining techniques that have been proposed in the past. Most of the research has been limited to detecting emotion at sentence level using annotated data. Support Vector Machine(SVM), Rule-based methods, Ensemble techniques are one of the majorly used methods by researchers. The survey also reviewed existing resources for facilitating emotion mining in English, including annotated datasets and lexicons. Lexicons like [Strapparava and Valitutti, 2004, Pennebaker et al., 2001, Mohammad and Turney, 2010a] include words and their corresponding emotion labels. Such resources provide explicit knowledge about type of emotion carried by words and phrases. These resources provide a kind of prior information to the emotion detection model and the model can perform well even in case of absence of sufficient amount of annotated data. Research for emotion mining in English enjoys rich resources that have been created by researchers over the years.

Hindi is a relatively resource-poor language. We do not find any other emotion mining work in Hindi, except [Ahmad et al., 2020]. They manually annotated a small dataset of Hindi news headlines with 9 class labels following the emotion theory proposed by [Plutchik, 1984]. They leveraged the emotion classification datasets available in English by utilising four transfer learning techniques. They first pre-

trained their deep learning model using two English emotion classification datasets and one Hindi sentiment classification dataset and then fine-tuned this model by using their own annotated dataset. For pre-training the model with English datasets, authors used bilingual embeddings as proposed in [Smith et al., 2017]. The deep learning model architecture that authors used in this work, comprises of a bidirectional LSTM layer, followed by three convolutional neural networks(CNNs). All the 3 CNNs have 200 filters each. The convoluted features from the three networks are then passed to a fully connected multi-layer perceptron(100 units), whose output is finally passed to an output layer which classifies the input into one of the nine classes. The model architecture is quite complex and contains huge number of parameters to be learnt. The dataset that they have released is not sufficient enough to properly learn the parameters of the model and the model will always underperform. They achieve performance improvements by employing cross-lingual and transfer learning approaches, but the scope for further improvements still remains. In this work, we use a simple LSTM model, which has much fewer parameters to learn.

2.2 Learning bilingual embeddings

One of the approaches to learn bilingual embeddings was introduced by [Mikolov et al., 2013b], where authors proposed a method to align the word embeddings spaces of two different languages by finding a linear transformation from one space to the other using a small bilingual dictionary.

Given a bilingual dictionary in 2 languages $L1$ and $L2$ of size n , let the word vector representations of these n word pairs be : $\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$, where $x_i \in \mathbb{R}^{d1}$ is the word vector corresponding to the i th word in source language($L1$) and $y_i \in \mathbb{R}^{d2}$ is the word vector corresponding to the i th word in target language($L2$)(translation of i th word in $L1$ to $L2$). The goal is to find a linear transformation W from $L1$ to $L2$ such that

$$\sum_{i=1}^n ||y_i - Wx_i||^2 \quad (2.1)$$

is minimised.

[Smith et al., 2017] takes this work forward and prove that this transformation W should be orthogonal. The transformation matrix W is now learnt by optimising the loss function with an additional constraint:

$$\max \sum_{i=1}^n y_i^T W x_i \quad (2.2)$$

subject to $W^T W = I$.

Here, if we take x_i and y_i to be unit vectors, then $y_i^T W x_i$ represents the cosine similarity of y_i and $W x_i$, under the transformation W , where x_i in $L1$ is transformed to $W x_i$ in $L2$. They show that above function can be optimised by decomposing W using Singular Value Decomposition, $W = UV^T$. Now, each vector x_i in space of $L1$ can be mapped to the space of $L2$, where it is calculated as $t_i = UV^T x_i$.

The authors have made available these transformation matrices W for mapping different languages on the shared space with English embeddings. We will particularly use the matrix corresponding to Hindi language(which will map Hindi and English embeddings on a shared space) in our experiments, as will be discussed in 6.1.2.

2.3 Learning task-specific embeddings

For Twitter sentiment classification, the authors in [Tang et al., 2014] tried to incorporate sentiment information besides syntactic context of words into the continuous word representations. They extended word embedding algorithm [Collobert et al., 2011] and trained three neural networks to learn Sentiment Specific Word Embeddings(SSWE). Then, SSWE are used in a supervised learning framework.

For emotion classification, [Agrawal et al., 2018] learnt Emotion Word Embeddings(EWE) starting from generic embeddings of GloVe [Pennington et al., 2014] and Word2vec [Mikolov et al., 2013a]. They tuned embeddings using a large corpus annotated by them, by backpropagating and updating embedding layer using LSTM network. They showed that using EWE outperformed generic embeddings.

In [Sakuma and Yoshinaga, 2019], authors proposed learning of task-specific embeddings for resource-poor(target) languages using resource-rich(source) language like English. Utilising general cross-lingual embeddings in source and target languages and task-specific embeddings of source language words, task-specific embed-

dings in target language were trained using two methods : linear projection and locally linear mapping(which is essentially a non-linear dimensionality reduction method). They explore locally linear embeddings to preserve the local topology across the embedding spaces, before and after the projection.

Chapter 3

Machine Learning

3.1 Optimisation Strategies

Machine Learning models use optimisation algorithms to search for the optimum values of model parameters(weights and biases) which minimise the value of the loss function. The search begins with some initial values of model parameters(values with which the model is initialised), at which point, the loss function value may be very high. The training procedure iteratively changes the values of model parameters, so as to reduce the loss at every step.

We discuss two of the most popular strategies used for optimisation.

3.1.1 Gradient Descent

This is inspired from [Deisenroth et al., 2020].

To reach the minimum value of the loss function, the parameters take steps proportional to the negative of the gradient of the loss at the current point(in the direction of the *steepest descent*) to quickly converge. The size of steps taken is decided by the amount of **learning rate**. A small learning rate will cause the parameters to take small steps(taking long time to converge) and a high learning rate can cause oscillations at the minima, with a possibility of never converging, thus it's important to carefully choose the value of η or use adaptive learning rate. Gradient descent update of a parameter w from time(say epoch) t to $t + 1$ is mathematically

written as:

$$w_{t+1} = w_t - \eta \nabla w_t \quad (3.1)$$

where, η is the learning rate, $\eta \in (0, 1)$ and ∇w_t is the gradient of the loss function w.r.t weight parameter at time t w_t .

3.1.2 Momentum based Gradient Descent

When gradient is in same direction for a few steps, then confidence is gained and a large step can be taken in the same direction in further epochs. To implement this strategy, instead of subtracting $\eta \nabla w_t$ from w_t at each update, v_t is subtracted which is calculated as shown in following update rule :

$$v_t = \gamma v_{t-1} + \eta \nabla w_t \quad (3.2)$$

$$w_{t+1} = w_t - v_t \quad (3.3)$$

here, η is momentum and $\gamma \in (0, 1)$. γv_{t-1} is a kind of history component. This history component v_t contains this history component and most recent gradient ∇w_t weighted by η . v_t is the exponentially decaying weighted sum of gradients of loss w.r.t weight parameter, as it can be expressed as:

$$v_t = \gamma^{t-1} \eta \nabla w_1 + \gamma^{t-2} \eta \nabla w_2 + \gamma^{t-3} \eta \nabla w_3 + \dots + \gamma \eta \nabla w_{t-1} + \eta \nabla w_t \quad (3.4)$$

Since $\gamma \in (0, 1)$, older gradients ∇w contribute less to v_t and recent updates have bigger contribution. Momentum based GD speeds up the updates in regions where loss function has gentler slope and it converges faster with less oscillations as compared to vanilla GD.

3.1.3 Frequency of Update of Parameters

Here, we discuss how frequently the weight and bias parameters of the model are updated during the training process. There are 3 strategies for when to update the model parameters.

Batch update

In batch update, all the samples of the training set are passed through the model, individual gradients are accumulated and at the end of seeing the complete training set, parameters are updated. Thus, there is one update in each epoch.

Mini-batch updates

In this type, training data is divided into batches of a fixed size. Say, we choose a batch size of 32, 32 samples of training set will be passed to the model at a time, and the update will happen at the end of seeing this batch. Thus, in each epoch, total number of updates are $\lceil \text{training set size} / \text{batch size} \rceil$. This is expected to converge faster than Batch update, as, parameters are updated more frequently, after seeing only few samples. This strategy is most widely used.

Stochastic update

This is a special case of Mini-batch update with batch size=1, i.e. parameters are updated after each training sample is passed through the model. Thus, in each epoch, total number of updates equal the size of training set. This slows down the training process a little.

3.2 Non-linear Activation functions

The outputs of hidden layers in a neural network are passed through non-linear activation functions. Non-linear activation functions help to approximate the non-linearity in the function. Without non-linearity in the network, multiple hidden layers in a neural network are as good as just a single layer in the network. We will discuss more about this in Section 4.2, where we talk about Feed forward neural networks in detail. Some popular and commonly used non-linear activation functions are discussed below.

3.2.1 Logistic

Logistic function is a kind of Sigmoid function. It is expressed as :

$$f(x) = 1/(1 + e^{-x}) \quad (3.5)$$

Range of $f(x) \in [0, 1]$ Derivative of Sigmoid function is 0 when x is too large(positive or negative) since the function becomes flat in these areas(saturated areas). Thus, gradients may vanish when input is in these regions, which results into weights not getting updated and hence no learning. To avoid this, weight parameters should be initialised with small values between $[0, 1]$. It is computationally expensive to compute due to exponential term.

3.2.2 Tanh

Tanh or Tangent Hyperbolic is also a kind of Sigmoid function. It is expressed as :

$$f(x) = (e^x - e^{-x})/(e^x + e^{-x}) \quad (3.6)$$

Range of $f(x) \in [-1, 1]$ This function is also saturated at extreme ends(when input is a large positive or negative value), and similarly, derivative is zero in these regions, thus causing vanishing gradient problem. It is also computationally expensive to compute due to exponential term.

3.2.3 Rectified Linear Unit(ReLU)

It is expressed as :

$$f(x) = \max(0, x) \quad (3.7)$$

ReLU is not saturated in the positive region of input values($x > 0$) and causes same problem of vanishing gradient problem as faced by sigmoid and tanh functions. Its derivative is 1 when $x > 0$ and 0 else. It is easy to compute as no exponential term is involved.

3.2.4 Leaky ReLU

It is expressed as :

$$f(x) = \max(0.01x, x) \quad (3.8)$$

It is an extension of ReLU. It solves the problem of vanishing gradient as there is no saturated region for $x < 0$ unlike ReLU. It is easy to compute and is the most widely used activation function.

3.3 Loss Functions

A loss function estimates how far the model predictions are, from the true values. While training a model, we want to minimise the loss value on the unseen(test) data, by minimising the loss on training data(but not at the expense of overfitting the data). The model uses the loss value to perform backpropagation, i.e., find the gradient of loss w.r.t model parameters and update them using an Optimisation algorithm as discussed in 3.1.

3.3.1 Mean Squared Error(MSE)

Mean Squared Error or MSE is used for **Regression** tasks, where the model predicts numerical value(s). MSE is the mean of squared differences between the true and the predicted values over samples in the set. Consider, a regression task, where the model outputs a single value(p_i). The model is being trained on the training set containing n samples, where, the true output values of samples are $y_1, y_2, \dots, y_n$ and the corresponding predicted output values are $p_1, p_2, \dots, p_n$.

Then, MSE on training set will be :

$$MSE_{training} = \frac{1}{n} \sum_{i=1}^n (y_i - p_i)^2 \quad (3.9)$$

3.3.2 Cross Entropy

Cross entropy is used to quantify the difference between a true probability distribution y and an estimated probability distribution p . In Machine Learning, Cross entropy is used as a loss function for **Classification** tasks. $y(x)$ is the true probability distribution for a sample x , over the classes in the dataset and $p(x)$ is the estimated probability distribution for x over the classes. $y(x)$ is *usually* a one-hot vector, with index 1 corresponding to the class to which sample x belongs and 0 for remaining classes. $p(x)$ contains probability values of sample x belonging to

different classes, as estimated by the model. For discrete probability distributions, y , and p , cross entropy is defined as :

$$H(p, q) = - \sum_x y(x) \log(p(x)) \quad (3.10)$$

Binary cross entropy

For categorical distribution over 2 classes(say, A and B), where, if the probability for a sample belonging to class A is p , then probability for that sample belonging to class B is $1 - p$. Considering the true distribution for a sample o to be y and $1 - y$ for classes A and B respectively and the estimated probability distribution to be p and $1 - p$, then Binary cross entropy loss is calculated as :

$$loss = -y \log(p) - (1 - y) \log(1 - p) \quad (3.11)$$

Categorical cross entropy

For classes more than 2, say, M number of classes, categorical cross entropy loss for a sample o is calculated as :

$$loss = - \sum_{c=1}^M y_{o,c} \log(p_{o,c}) \quad (3.12)$$

Here, $y_{o,c}$ and $p_{o,c}$ are respectively, the actual probability and the predicted probability of a sample o belonging to class c .

3.3.3 Softmax function

The raw output values of a neural network ought to be converted to probability distribution values, to apply the loss functions discussed above. Consider, the output layer of a neural network has size m , i.e., the dataset on which the network is being trained has m classes. The network outputs m raw values, $y_1, y_2, \dots, y_m$. The probability distribution of a training sample belonging to different classes is calculated by applying softmax function on the raw output values. Thus, probability of a training sample belonging to the i th class, p_i is calculated as :

$$p_i = \frac{e^{y_i}}{\sum_{j=1}^m e^{y_j}} \quad (3.13)$$

Similarly, all the probabilities, p_i for $i = 1$ to m are calculated. These values satisfy the condition :

$$\sum_{i=1}^m p_i = 1 \quad (3.14)$$

since, the probability distribution values over the classes should add up to 1.

3.4 Evaluating Models

For classification tasks, most commonly used metrics for measuring the performance of an ML(Machine Learning) model are Precision, Recall and F1 score. Before talking about these three in detail, we first need to understand the concept of a positive and a negative class.

Consider a classification task of finding *Credit card defaulters*, the customers who will most likely default the credit card payment. This is a binary classification task, i.e., a customer in a bank can belong to one of the 2 categories : either the customer will default or not. Since, the bank is more interested to find who will default the credit card payment, the **positive** class is *credit card defaulters*, and the **negative** class is *Non credit card defaulters*. There are many such applications like this, where one class is of more importance(*positive* class) to the stakeholders than the other(*negative* class). So, we are more interested to know how our model performed for the positive class, because the misclassification cost for this class will be much higher than the misclassification cost for the negative class.

Accuracy metric weighs both positive and negative classes equally, so it is inappropriate to use in many applications.

We also need to understand the following terminologies :

1. **True Positive(TP)** : samples which were *correctly* labelled as belonging to the **Positive** class. This value should be maximised.
2. **False Positive(FP)** : samples which were *incorrectly* labelled as belonging to the **Positive** class. It's a misclassification(**Type I error**) and the samples actually belong to the negative class.
3. **True Negative(TN)** : samples which were *correctly* labelled as belonging to the **Negative** class.

4. **False Negative(FN)** : samples which were *incorrectly* labelled as belonging to the **Positive** class. It's a misclassification(**Type II error**) and the samples actually belong to the positive class. This value should be minimised.

3.4.1 Precision

Precision is defined as the number of samples which were correctly classified as belonging to the positive class(TP), out of all the samples labelled as positives(TP + FP).

$$Precision = \frac{TP}{TP + FP} \quad (3.15)$$

3.4.2 Recall

Recall is defined as the number of samples which were correctly classified as belonging to the positive class(TP), out of all the actual positive samples(TP + FN).

$$Recall = \frac{TP}{TP + FN} \quad (3.16)$$

3.4.3 F1 score

F1 score is the **Harmonic mean** of Precision and Recall. Mathematically,

$$F1score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (3.17)$$

3.4.4 Averaging the metrics over multiple classes

For tasks with multiple classes(more than 2), we need to calculate the average metrics, Average Precision, Average Precision and Average F1 score. Consider, there are 3 classes : c_1, c_2, c_3 . To find average values, 3 binary classifications are performed, with each class as the positive class once. Say, firstly, if we take c_1 as the *positive* class, then the *negative* class will be **not** c_1 . For this classification, metrics will be calculated and then metrics will be further calculated taking c_2 and c_3 as positive classes. There are two ways to find the average metrics:

Macro-averaging

Taking the above case of 3 classes, let's call Precision values P_1, P_2, P_3 with c_1, c_2, c_3 as the positive class respectively. Similarly, recall values, R_1, R_2, R_3 and F1 scores, F_1, F_2, F_3 . Then,

$$MacroPrecision = \frac{P_1 + P_2 + P_3}{3} \quad (3.18)$$

$$MacroRecall = \frac{R_1 + R_2 + R_3}{3} \quad (3.19)$$

$$MacroF1score = 2 \cdot \frac{MacroPrecision \cdot MacroRecall}{MacroPrecision + MacroRecall} \quad (3.20)$$

Micro-averaging

Taking the same case of 3 classes, call $TP_1, TP_2, TP_3, FP_1, FP_2, FP_3$ and FN_1, FN_2, FN_3 as True Positive values, False Positive values and False Negative values for the 3 classes positively. Then,

$$MicroPrecision = \frac{TP_1 + TP_2 + TP_3}{TP_1 + TP_2 + TP_3 + FP_1 + FP_2 + FP_3} \quad (3.21)$$

$$MicroRecall = \frac{TP_1 + TP_2 + TP_3}{TP_1 + TP_2 + TP_3 + FN_1 + FN_2 + FN_3} \quad (3.22)$$

$$MicroF1score = 2 \cdot \frac{MicroPrecision \cdot MicroRecall}{MicroPrecision + MicroRecall} \quad (3.23)$$

Chapter 4

Artificial Neural Networks

This chapter is mainly inspired from [Goldberg, 2017, Goodfellow et al., 2016]

4.1 Inspiration from biological neurons [Jain et al., 1996]

Artificial Neural Networks or ANNs are inspired from how the biological network of neurons in human brain process information. **Neuron** is a biological cell in human brain which is responsible for processing information. A neuron receives signals from other neurons through its *dendrites*(receivers) and transmits the signals that it produces in its cell body along its *axon*(transmitters). Between the axon strand of one neuron and dendrite strand of other neuron, lies a functional unit called **synapse**. When a signal reaches synapse's terminals, chemicals called *neurotransmitters* are released. Neurotransmitters enhance or inhibit receptor neuron's tendency to emit electrical impulses by diffusing into synaptic gap. The synapse's effectiveness can be adjusted by the signals passing through it, so synapses can learn from the activities in which they participate. This dependence on history acts as memory and is possibly responsible for human memory. Cerebral cortex in human brain contains approximately 10^{11} neurons and 10^{14} - 10^{15} neuron interconnections.

ANNs are very capable computational devices and function approximators. In ANN, a neuron is a computation unit that is connected to other neurons with weighted connections. A neuron essentially performs weighted sum of its inputs,

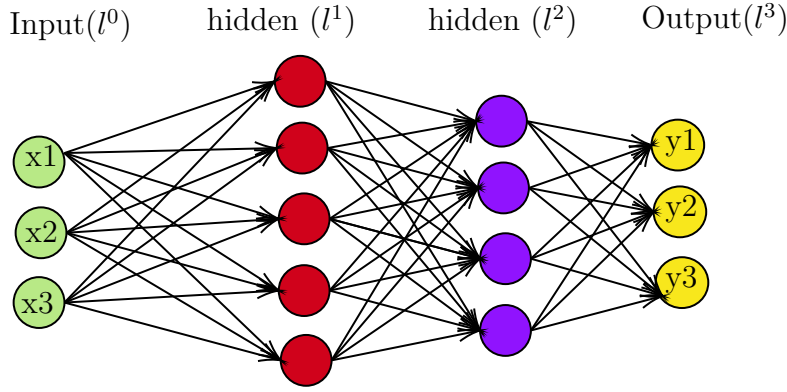


Figure 4.1: Feed Forward Neural Network

applies a non-linearity and outputs the result to other neuron. An ANN, in essence, is a weighted graph with neurons connected to each other. A neuron takes input from one or more neurons and outputs the result to one or more neurons.

4.2 Feed Forward Neural Networks

A typical feed forward neural network is as shown in fig.4.1. It consists of Input layer : l^0 (dimension=3), hidden layer 1 : l^1 (size=5), hidden layer 2 : l^2 (size=4) and Output layer : l^3 (dimension=3). All the neurons in layer l^n are connected with all the neurons in layer l^{n+1} , where $n = 0, 1, 2$. This is called a *fully connected layer* or an *affine layer*. A full layered connection, say from l^0 to l^1 can be thought of as a **linear transformation** from 3 dimensions to 5 dimensions, so there is a matrix associated with each layer (called *weight matrix*). Element-wise non-linearity such as *tanh*, *relu*, *leaky relu* or *sigmoid* is added to the output vector after each layer. Adding non-linearity helps to clip the output values between a defined range and it approximates non-linear mappings from input to output. Without non-linearity, if the network consists of only linear transformations, then multiple layers in the network is equivalent to just 1 layer, as, multiplication of linear transformations (matrices) is a linear transformation (matrix) only. Layer l^1 outputs a row vector $h_1 \in \mathbb{R}^5$, $h_1 = xW_1$, where x is input row vector $x \in \mathbb{R}^3$ and $W_1 \in \mathbb{R}^{3 \times 5}$. Then, adding non-linearity to h_1 , $h_1 = g(h_1)$ where $g(\cdot)$ is any non-linear function as discussed above. Similarly, layer l^2 outputs row vector $h_2 \in \mathbb{R}^4$, $h_2 = g(h_1W_2)$, where $W_2 \in \mathbb{R}^{5 \times 4}$. Finally, output layer l^3 gives output row vector $o \in \mathbb{R}^3$, $o = h_2W_3$,

where $W_3 \in \mathbb{R}^{4 \times 3}$. Raw values of output vector o can be used to calculate the loss, or softmax can be applied to find probability distribution over the output values (in case of classification task). Depending upon the loss function being used, any other non-linear function can also be used after output layer (say softmax to limit the values between 0 and 1).

4.3 Sequence Modeling

Sequence models are useful for processing sequential data like text (text is a sequence of sentences), time-series data, music, audio etc. Sequence models help to remember context in data being processed, like in natural language understanding tasks, it's important to keep context in mind to fully understand the meaning of text.

Traditional neural networks cannot persist context and treats every input as independent of others. When dealing with languages, we need to retain some past information, that we have encountered so far. Thus, to fully understand the meaning of any sentence, we require contextual knowledge, which, traditional neural networks cannot provide. Sequence models overcome this shortcoming by introducing a recurrent layer which allows information to persist. Recurrent layers take input at each timestep, and produce output by multiplying the input with a set of weights which is shared across all timesteps.

In this section, we discuss various models which are useful for processing sequential data.

4.3.1 Recurrent Neural Networks(RNN)

RNNs can approximate any function involving recurrence. They represent arbitrary sized sequential inputs as fixed-size vectors (often called as *context vector*), which are often lossy. Depending upon how the network is trained, this lossy summary output might keep some aspects of the input data more precisely than some other aspects. RNN architecture represented in 4.2 (inspired from figure 10.5 in [Goodfellow et al., 2016]) takes as input a temporal sequence of size T . The network is designed for tasks like emotion detection (or any other classification task), with a single output predicted at the end of the reading the entire sequence. At the end, loss is computed,

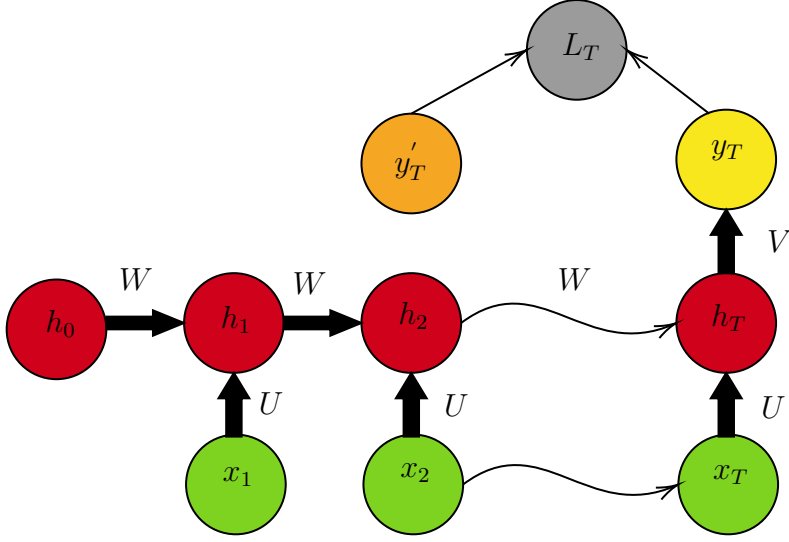


Figure 4.2: Recurrent Neural Network

which, in this case is categorical cross entropy.

Following set of equations mathematically represent an RNN architecture like 4.2.

$$a_{(t)} = b + \mathbf{W}h_{(t-1)} + \mathbf{U}x_{(t)} \quad (4.1)$$

$$h_{(t)} = \tanh(a_{(t)}) \quad (4.2)$$

$$o_{(T)} = c + \mathbf{V}h_{(T)} \quad (4.3)$$

$$y'_{(T)} = \text{softmax}(o_{(T)}) \quad (4.4)$$

where, $\mathbf{U}$, $\mathbf{V}$ and $\mathbf{W}$, respectively, are the weight matrices for input-to-hidden, hidden-to-output, hidden-to-hidden connections.

Hidden state at current timestep t , $h_{(t)}$, is calculated by taking as input 2 values : hidden state at previous timestep $t - 1$, $h_{(t-1)}$ (associated weight matrix W) and input data at current timestep t $x_{(t)}$ (associated weight matrix U) and then adding non-linearity $\tanh$. It is calculated till $t = T$. Finally, at last timestep, $t = T$, $o_{(T)}$ is calculated from $h_{(T)}$ (associated weight matrix V) and then $y'_{(T)}$ is calculated by applying softmax to find probability distribution over class labels, which is used to find loss value.

4.3.2 Long Short Term Memory(LSTM)

RNNs suffer from **vanishing** gradient(mostly) and **exploding** gradient problems while trying to capture long-term dependencies. Gradients tend to vanish when backpropagating errors over very long sequences, which results into forgetting of long-term dependencies, and, when they explode, the learning becomes quite unstable. Also, all of the past history is not equally important to be remembered. We can forget some non-essential information and retain some important knowledge, both of which depend on the task at hand. This is what LSTMs do.

Originally proposed by [Hochreiter and Schmidhuber, 1997], LSTMs learn what to forget and what to remember from the past history using a gating mechanism. The most popular LSTM architecture is described below.

$$i = \sigma(x_j W^{x_i} + h_{j-1} W^{h_i}) \quad (4.5)$$

$$f = \sigma(x_j W^{x_f} + h_{j-1} W^{h_f}) \quad (4.6)$$

$$o = \sigma(x_j W^{x_o} + h_{j-1} W^{h_o}) \quad (4.7)$$

$$z = \tanh(x_j W^{x_z} + h_{j-1} W^{h_z}) \quad (4.8)$$

$$c_j = (f \odot c_{j-1} + i \odot z) \quad (4.9)$$

$$h_j = o \odot \tanh(c_j) \quad (4.10)$$

where,

- $\odot$ is the Hadamard Product(element-wise multiplication)
- i, f, o are input, forget and output gates respectively.
- $W^{x_i}, W^{h_i}, W^{x_f}, W^{h_f}, W^{x_o}, W^{h_o}, W^{x_z}, W^{h_z}$ are model parameters to be learnt
- x_j is the input at timestep j
- c_j, h_j are the cell state and hidden state respectively at timestep j
- c_{j-1}, h_{j-1} are the cell state and hidden state respectively at timestep $j - 1$
- f controls how much of old memory c_{j-1} will be forgotten and z controls how much of input i will contribute to new memory c_j

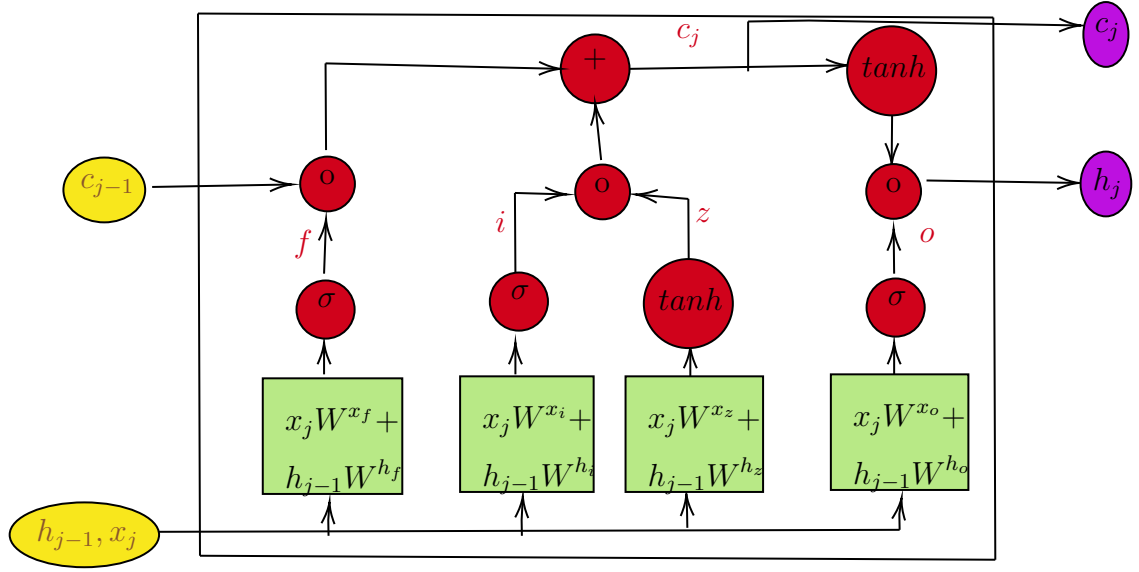


Figure 4.3: an LSTM cell

- Finally, h_j is the combination of output gate o and current cell state or memory c_j

The figure 4.3 (inspired from ¹) shows an LSTM cell at timestep j . This LSTM cell is what is represented by the equations above. At timestep j , the cell takes h_{j-1}, x_j, c_{j-1} as input and outputs h_j, c_j .

¹<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Chapter 5

Learning Word Embeddings

Input to neural models requires words to be represented as numerical values. Representation of words as one-hot vectors is simple, but, has many drawbacks. Size of these vectors is very large(size of vocabulary, which can be in millions), and these are sparse vectors(all zeros except one index, which is one). Moreover, these representations donot tell anything about the similarity between words which often occur together in similar contexts and thus, provide little to no knowledge. In this section, we summarise few popular algorithms that effectively learn word representations from large corpus of data(containing billions of words and millions of unique words) using neural network architectures. These representations are dense vectors with very low dimensions(as compared to dimension of one-hot vectors), and vectors of similar words occur close together in the vector space representing syntactic regularities in the language.

5.1 Word2Vec

Two neural network model architectures for learning word representations were proposed by [Mikolov et al., 2013b] : Continuous Skip-gram(popular as simply *Skip-gram*) and Continuous Bag of Words(popular as *CBoW*). In [Mikolov et al., 2013c], same authors proposed extensions to these architectures to make training, more time and computationally efficient and improve accuracy.

5.1.1 Continuous Skip Gram

Skip gram takes every word in the corpus as input and tries to maximise the probability of predicting its context words(words surrounding the input word) in the context window. In the fig.5.1, size of context window is 2, so 2 words to the left of w_t : w_{t-1} and w_{t-2} and 2 words to the right of w_t : w_{t+1} and w_{t+2} are considered as output.

The probability of a word in the vocabulary being a context word(w_{t+j}) of the input(w_t) is calculated using softmax value given by:

$$p(w_{t+j}|w_t) = \frac{e^{v_{w_t}^T \cdot v_{w_{t+j}}}}{\sum_{w_i \in V} e^{v_{w_t}^T \cdot v_{w_i}}} \quad (5.1)$$

where, v_{w_t} is the vector corresponding to the input word w_t and $v_{w_{t+j}}$ is the vector corresponding to the context word w_{t+j} , where j is the size of context window(indices of context words ranging from $t - j$ to $t + j$). Denominator of the equation is computationally expensive to solve.

The objective function is to maximise the sum of probabilities of words being the context words given a center word :

$$J_\theta = \frac{1}{T} \sum_{t=1}^T \sum_{-n \leq j \leq n, j \neq 0} \log p(w_{t+j}|w_t) \quad (5.2)$$

5.1.2 Continuous Bag of Words(CBoW)

CBoW predicts a center word(w_t), given the context words($w_{t-2}, w_{t-1}, w_{t+1}, w_{t+2}$), as shown in the fig.5.2. Input to the model is a *bag of words*, where, the order of the words does not matter.

The objective is to maximise the probability of predicting the center word given the context words, with context window of size n :

$$J_\theta = \frac{1}{T} \sum_{t=1}^T \log p(w_t|w_{t-n}, w_{t-n+1}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+n-1}, w_{t+n}) \quad (5.3)$$

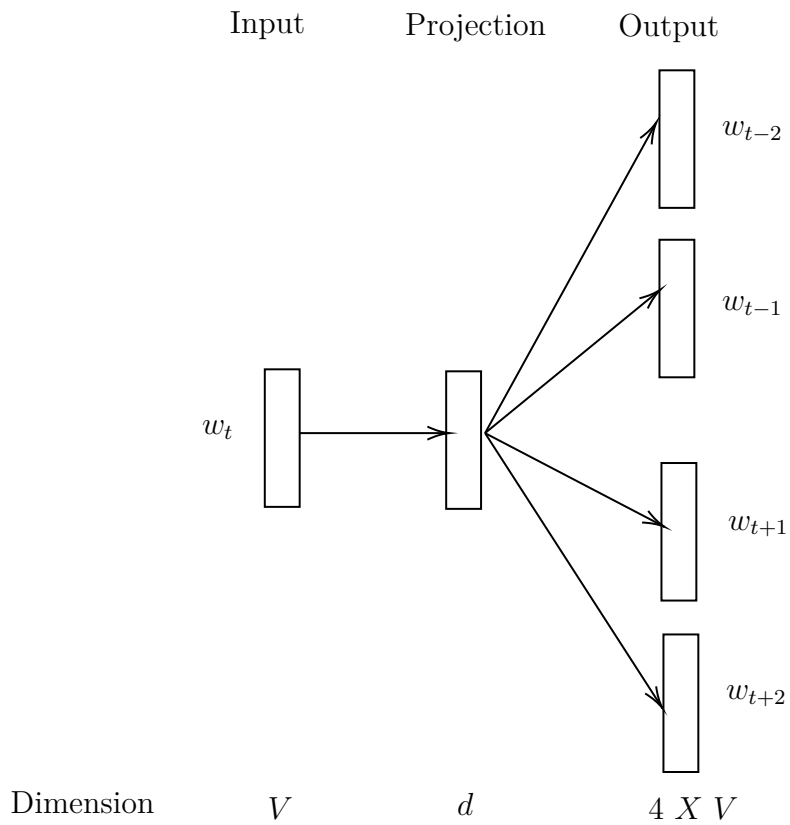


Figure 5.1: Continuous Skip Gram

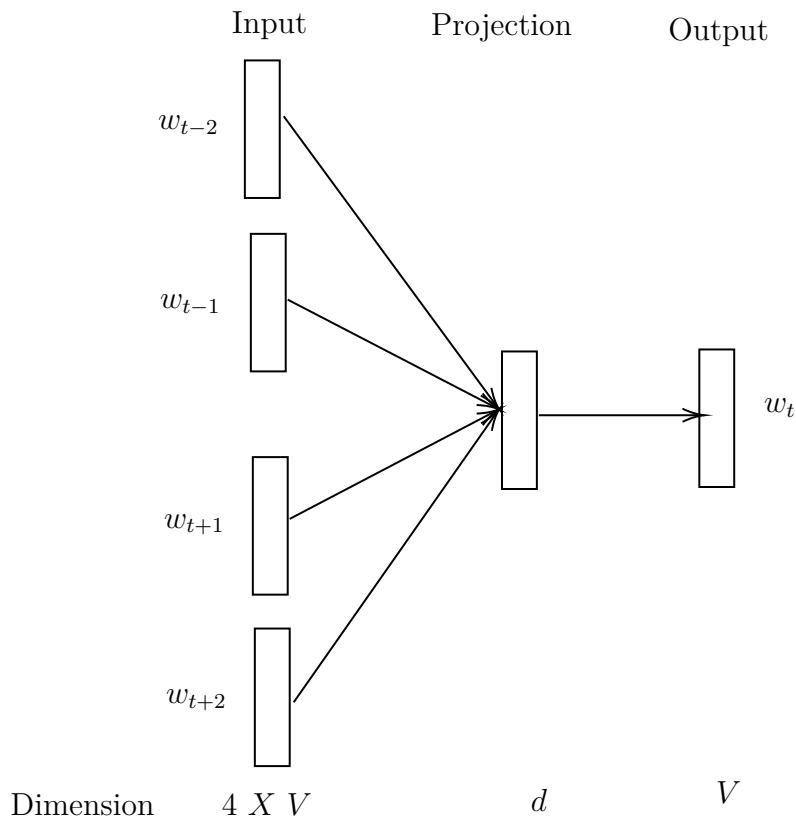


Figure 5.2: Continuous Bag of Words

5.2 FastText

Fasttext algorithm was proposed by [Bojanowski et al., 2017] by Facebook AI Research Team. It takes into consideration sub-word information. Based on skip-gram approach, vector representation of a word is formed by summing up the vector representations of bag of character n-grams. This approach allows to compute vector representations for Out of Vocabulary(OoV) words(words that never appeared during the training process).

According to [Bojanowski et al., 2017], the scoring function is obtained by :

$$s(w, c) = \sum_{g \in G_w} z_g^T v_c \quad (5.4)$$

where, G_w is the set of all n-grams of word w , for $3 \leq n \leq 6$; z_g is the vector representation of each character n-gram and v_c is the vector representation of the context word.

Chapter 6

Proposed Work

6.1 Training models for Emotion mining in Hindi

6.1.1 Model I : Baseline Model

For our baseline model, we use LSTM model which we discussed in 4.3.2. The model architecture is as shown in fig.6.1 The LSTM network consists of 1 hidden layer containing 100 hidden units. The output layer consists of 9 units corresponding to 9 classes in the dataset. An embedding matrix is added to the input layer, which we jointly refer to as embedding layer further.

We take mini-batch size of 64 and maximum sequence length(number of words in sentence) of the input sentences as 100. The sentences with length > 100 are

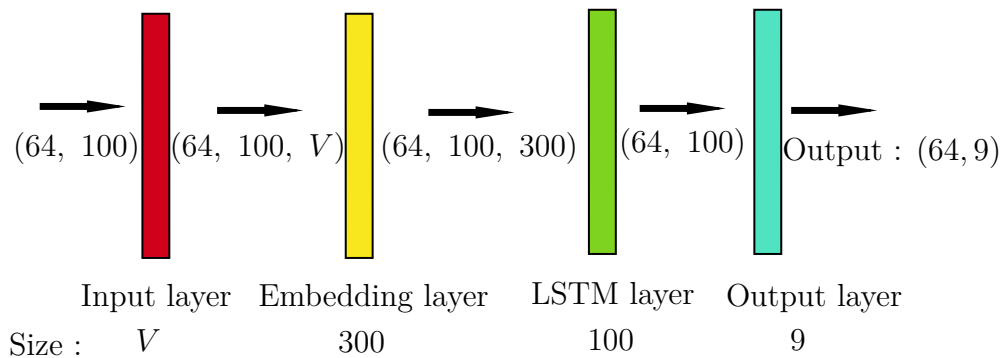


Figure 6.1: Model I(Baseline) Architecture

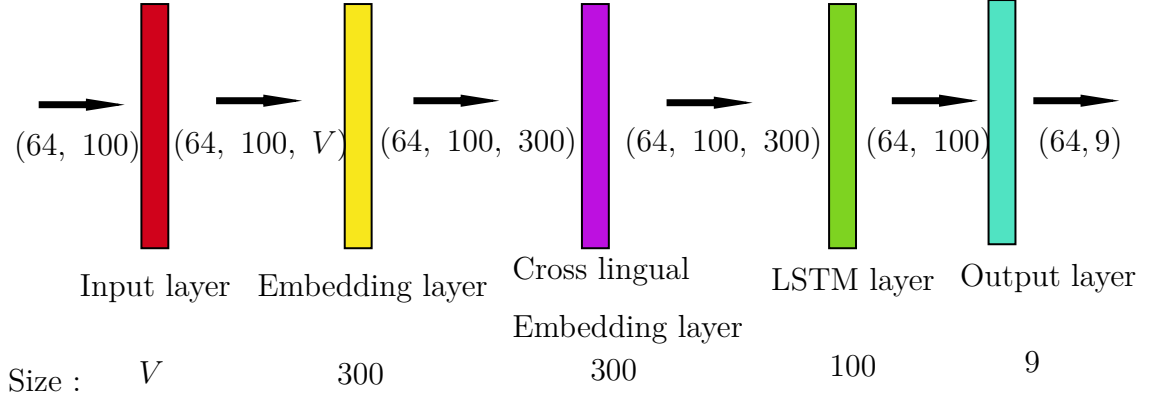


Figure 6.2: Model II(Cross-lingual) Architecture

truncated from the end and sentences with length < 100 are padded with zeros at the end. So, a batch of input, becomes $(64, 100, V)$, where, initially, each word in the sentence is represented as a one-hot vector of size V (size of vocabulary). After passing through the embedding layer(dim=300), the data size becomes $(64, 100, 300)$. Since the hidden layer size of LSTM is 100, the combined output of LSTM layer becomes $(64, 100)$. Finally, the output layer gives softmax probability distribution values over the nine classes for each input sentence in the batch.

This embedding matrix is initialised in following way.

using Fasttext embeddings

Fasttext embeddings [Bojanowski et al., 2017] represent a word by the sum of vector representations of its character n -grams and the vector representation of the word itself. Character n -grams are formed for $3 \leq n \leq 6$. It allows sharing the representations across words, thus allowing to learn reliable representations for rare and Out of vocabulary words. We use pre-trained Fasttext embeddings² as proposed in [Grave et al., 2018]

6.1.2 Model II : Cross-lingual transfer

To improve upon our baseline model, we perform cross-lingual transfer using knowledge from rich supervised English resources. We first pre-train our model using

²Available at : <https://fasttext.cc/docs/en/crawl-vectors.html>

English dataset Semeval labelled with emotion categories. Then we fine-tune the model using Hindi emotion-labelled dataset.

The Hindi emotion dataset is labelled with 9 classes. Although there are only 4 classes in Semeval dataset, to keep the number of classes in the output layer same (in the pre-trained and the fine-tuned models), we pre-train the model with 9 classes in the output layer. The model architecture is same as used in Model I. The embedding layer consists of English monolingual Fasttext embeddings. Parameters in LSTM layer and output layer are learnt using backpropagation, keeping the embedding layer fixed. This allows the model to capture task-specific features in LSTM layer and in output layer (for 4 classes - joy, sadness, anger and joy). We expect to transfer these task-specific parameters from English to Hindi using English-Hindi bilingual embeddings.

The embedding matrix for this model is initialised in following way.

Bilingual embeddings

We discussed in 2.2, how authors in [Smith et al., 2017] learnt transformation matrix W to map embeddings of 2 languages on a shared space. They mapped English and Hindi embeddings on the same space using English-Hindi bilingual dictionary and monolingual English and Hindi Fasttext embeddings, these mapped embeddings are **bilingual embeddings**. Only the Hindi monolingual embeddings need to be transformed, by multiplying with the transformation matrix. The matrix will take a Hindi monolingual embedding and map it onto the space of English monolingual Fasttext embeddings, to obtain Hindi bilingual embedding. The transformation matrices for many languages (with English as reference language) are available at ³.

Given a monolingual Hindi embedding $e_{mono} \in R^{300 \times 1}$ and transformation matrix $T^{300 \times 300}$, Hindi bilingual embedding is obtained as :

$$e_{bi} = T e_{mono} \quad (6.1)$$

where, $e_{bi} \in R^{300 \times 1}$.

To fine-tune the model with Hindi dataset, we do the following:

³https://github.com/babylonhealth/fastText_multilingual

- add cross-lingual embedding layer, after the embedding layer of Hindi monolingual Fasttext embeddings, as shown in fig.6.2(compare this with fig.6.1)
- this layer will take Hindi monolingual Fasttext embeddings(of size (64, 100, 300)) and convert them into Hindi bilingual embeddings(size=(64, 100, 300)).
- load the learnt parameters of LSTM layer and output layer from the pre-trained model

6.1.3 Model III : Learning a mapping from monolingual to task-specific embedding space

A neural network approximates the function that maps the input data points to the corresponding output data points. We have two embedding spaces : English monolingual(Fasttext embeddings) and English task-specific(affective embeddings). In [Agrawal et al., 2018], authors learnt **task-specific embeddings** for the task of emotion mining, or **affective embeddings**⁴, as they call them. Now, we want to learn a mapping W from English monolingual embedding space to English task-specific embedding space. Using W and Hindi monolingual embeddings(Fasttext embeddings), we wish to approximate Hindi task-specific embeddings.

So, given English monolingual embeddings, X^{en} , English task-specific embeddings, Y^{en} , and, Hindi monolingual embeddings, X^{hi} , we want to approximate Hindi task-specific embeddings, Y^{hi} . Below, we discuss the neural network methods of learning linear and non-linear mappings.

Linear mapping

This method was also used by authors in [Sakuma and Yoshinaga, 2019] to learn linear projection. To learn a linear mapping from a given monolingual embedding space, S_1 , to a given task-specific embedding space, S_2 , we need to find a matrix, $W \in \mathbb{R}^{d \times d}$ which transforms a column word vector $u \in \mathbb{R}^d$ in S_1 to the column word vector $v \in \mathbb{R}^d$ in S_2 , such that,

$$v = Wu \tag{6.2}$$

⁴Available at : https://www.dropbox.com/s/5egqnbktbfxp2im/ewe_uni.txt.zip?dl=0

The neural network architecture to learn such a mapping, contains just the input and the output layers, which are both 300 dimensional, in this case. We make pairs of embeddings, by taking Fasttext embedding of a word w as the input vector(x) and the affective embedding of w as the output vector(y). First, we create the vocabulary of words, V for which the affective embeddings are available(since, this vocabulary is much smaller than the vocabulary of words having Fasttext embeddings), and then, pair up the Fasttext and affective embeddings of same words. We train transformation matrix W that maps English monolingual embeddings X^{en} to task-specific embeddings Y^{en} by estimating as:

$$\underset{W}{\operatorname{argmin}} \frac{1}{|V|} \sum_{i=1}^{|V|} \|WX_i^{en} - Y_i^{en}\|^2 \quad (6.3)$$

where, X_i^{en} and Y_i^{en} are the i th the vectors of X^{en} and Y^{en} respectively, and $|V|$ is the size of vocabulary V .

Now, to find Y^{hi} , X^{hi} (Hindi monolingual embedding)vectors are first multiplied with transformation matrix to map Hindi embeddings onto the same space as English embeddings as explained in 6.1.2, we call them Hindi bilingual embedding $X^{hi_{bi}}$. Finally, using W , Y^{hi} is estimated as:

$$Y^{hi} = WX^{hi_{bi}} \quad (6.4)$$

Let's call Y^{hi} learnt here, as Y_{lin}^{hi} .

Non-linear mapping

To learn the non-linear mapping, we introduce non-linearity by adding a hidden layer between input and output layers. Size of hidden layer is 100 units with Leaky ReLU as non-linear activation.

Taking X_i^{en} as the i th input vector, model estimates the corresponding output vector $Y_i'^{en}$ as:

$$h_i = W_1 X_i^{en} \quad (6.5)$$

$$Y_i'^{en} = W_2 h_i \quad (6.6)$$

where, $h_i \in R^{100}$ is the output column vector of hidden layer. W_1 weight matrix is of size $R^{100 \times 300}$ and W_2 weight matrix is of size $R^{300 \times 100}$.

Loss is calculated as mean of squared L2 norm of error :

$$\frac{1}{|V|} \sum_{i=1}^{|V|} \|Y_i'^{en} - Y_i^{en}\|^2 \quad (6.7)$$

where, Y_i^{en} is the i th true output vector.

The model parameters W_1, W_2 are learnt via Momentum based Stochastic Gradient Descent algorithm using backpropagation method.

Now, using these learnt model parameters and Hindi bilingual embeddings $X^{hi_{bi}}$, Hindi task-specific embeddings Y^{hi} are estimated by following the equations 6.5 and 6.6.

Let's call Y^{hi} learnt here, as Y_{non}^{hi} .

In 6.4.4, we will use Hindi bilingual task-specific embeddings Y_{lin}^{hi} and Y_{non}^{hi} in the embedding layer, for cross-lingual transfer model.

Training details

Total samples = 183712
Train-test split ratio = 80 : 20
Training samples = 146969
Test samples = 36743
Optimisation Algorithm : Stochastic Gradient Descent
Learning rate = 0.05
Epochs = 10

Table 6.1: Model Settings and Hyperparameters

6.2 Reference for comparisons

We report the following result tables from [Ahmad et al., 2020]. Table 6.2 contains the macro-averaged Precision, Recall and F1 scores for the baseline models using Hindi emotion dataset and English Semeval dataset, discussed in 6.3.

Dataset	Precision	Recall	F1 score
Hindi emotion	0.522	0.453	0.477
English Semeval	0.862	0.864	0.863

Table 6.2: Baseline Results

In table 6.3, the macro-averaged metrics are reported which the authors obtained in their work for cross-lingual transfer using different transfer learning techniques. Transfer learning techniques that they used are : AU(All Unfreeze), TU(Top-down Unfreeze), BU(Bottom-up Unfreeze) and GU(Gradual Unfreeze). These techniques differ in the way they freeze and unfreeze different layers in the model.

Strategy	Precision	Recall	F1 score
AU	0.581	0.482	0.514
TU	0.624	0.485	0.532
BU	0.573	0.473	0.508
GU	0.667	0.449	0.516

Table 6.3: Cross-lingual with different transfer learning techniques

In 6.4, we will compare our obtained results with these results.

6.3 Description of datasets

In this section, we discuss about the datasets in detail, which we used in the experiments.

6.3.1 Hindi emotion dataset

In our experiments, we use Hindi emotion labelled dataset released by [Ahmad et al., 2020]. It consists of 3332 Hindi news headlines, each of which is labelled with one of the 9 emotion classes. The 9 categories and the samples belonging to each category is as shown in Table 6.4

Category	Samples belonging to this category
Sadness	1849
Sympathy/Pensiveness	461
No-emotion	271
Optimism	218
Joy	180
Fear/Anxiety	176
Disgust	78
Anger	58
Surprise	41

Table 6.4: Hindi emotion dataset : Distribution of samples across categories

Dataset cleaning

All the digits, special characters like # , : ? ”” etc. and some English characters which maybe there, were removed.

6.3.2 English emotion dataset

[Mohammad et al., 2018] released the Task 1 : Affect in Tweets for SemEval-2018. We take the datasets from subtask Emotion Intensity Ordinal Classification(EI-oc). The task is to classify the given tweet and its emotion class(E) into one of 4 ordinal classes of intensity of E, that best represents the mental state of the tweeter. So, there is an ordinal number from 0 to 4 corresponding to each tweet and emotion class. Value 0 indicates that the tweet does not impart the emotion with which it is labelled. Therefore, we remove those tweets which have the intensity value of zero corresponding to them. The tweets in the training, development and test sets add up to 12,635. Removing such tweets(5429) from the set, we are left with a total of 7206 tweets, which we will finally use in the experiments. The 4 categories in the dataset and the samples belonging to each category is as shown in Table 6.5

Category	Samples belonging to this category
Anger	1996
Fear	1253
Joy	2214
Sadness	1743

Table 6.5: English emotion dataset : Distribution of samples across categories

Dataset cleaning

Since this dataset contains users' tweets, which may contain hashtags and emojis, so all of that data had to be removed. So, hashtags, URLs, emojis, punctuation marks and digits were removed. Then, all uppercase characters were converted to lowercase.

6.4 Experiments, Results and Analysis

All the experiments were performed in Python programming language⁵, using PyTorch, an open source machine learning framework⁶.

We performed 3 fold cross validation and report Macro-averaged metrics (Precision, Recall and F1 scores) over 3 folds.

6.4.1 Experiment I : Hindi baseline

In this experiment, we use the model described in 6.1.1. The model hyperparameters and settings are as listed in Table 6.6. We train different models with the different number of epochs. Precision, Recall and F1 scores in Table 6.7 are Macro-averaged over all the 9 classes.

⁵<https://www.python.org/>

⁶<https://pytorch.org/>

Layers = 1
Hidden layer size = 100
Optimisation Algorithm : Momentum based Gradient Descent
Learning rate = 0.05
Momentum = 0.5
Batch size = 64
Maximum sequence length = 100

Table 6.6: Model Settings and Hyperparameters

Epochs	Precision	Recall	F1
100	0.417	0.439	0.410
150	0.504	0.519	0.495
200	0.558	0.567	0.547
250	0.617	0.621	0.604

Table 6.7: Results I

Comparison and Analysis

For our baseline model, we obtain the best metrics as shown in the row with bold values, with the best F1 score of 0.604. Comparing this with the reported baseline F1 score of 0.477 in 6.2, we get significantly better results with our much simpler model and much fewer parameters to learn. Since, the dataset is insufficient to efficiently train a complex model with large number of parameters such as used in [Ahmad et al., 2020], the results that they obtain are not good.

6.4.2 Experiment II : Model Pre-training

In this experiment, we pre-train our model with English dataset, Semeval 2018 to perform cross-lingual transfer in the next experiment. We expect to transfer required task knowledge from rich English resources to improve our baseline model. Model settings and hyperparameters are same as specified in 6.6, except for batch size which is 128 for this experiment. Fasttext embeddings are used to initialise the embedding layer.

Epochs	Precision	Recall	F1
100	0.608	0.625	0.593
150	0.698	0.705	0.684
200	0.732	0.734	0.718
250	0.749	0.744	0.732

Table 6.8: Results II

We get the best metrics when the model is trained for 250 epochs, with the highest F1 score of 0.732(row with bold values). Thus, we save the weights of this model to perform cross-lingual transfer learning in 6.4.3.

6.4.3 Experiment III : Cross-lingual transfer

In this experiment, we fine-tune the model which we pre-trained with English dataset in Experiment II. Model settings and hyperparameters are same as specified in 6.6.

For this set of experiments, the model used is as described in 6.1.2 The model parameters are initialised with the weights of the model saved in Experiment II 6.4.2 and Hindi bilingual embeddings are used in the embedding layer.

Epochs	Precision	Recall	F1
100	0.616	0.613	0.599
150	0.642	0.639	0.624
200	0.653	0.648	0.635
250	0.639	0.633	0.621

Table 6.9: Results III

Comparison and Analysis

In this experiment, we see the best F1 score 0.635 is corresponding to the model which was trained for 200 epochs(row with bold values). This improves the baseline model, where we obtained an F1 score of 0.604 by around 3%.

Comparing this with the best F1 score of 0.532 as obtained by [Ahmad et al., 2020] in 6.3, we again surpass the results by a high margin of 10%. They trained and compared models using four different transfer learning techniques, which are

quite complex in the way they freeze, unfreeze and train different combination of layers. We train the entire network at once, following a standard backpropagation algorithm to update weights in all the layers.

6.4.4 Experiment IV : Using Hindi task-specific embeddings

In this set of experiments, we use Hindi task-specific embeddings in the embedding layer, which we discussed in 6.1.3.

Experiment IV.A

In this experiment, we use the model architecture described in 6.1.2 with Hindi task-specific embeddings as learnt using linear mapping in 6.1.3.

Epochs	Precision	Recall	F1
100	0.654	0.649	0.637
150	0.702	0.696	0.686
200	0.693	0.682	0.674
250	0.698	0.686	0.678

Table 6.10: Results IV.A

Discussion and Analysis

We obtain the best F1 score of 0.686, improving the best F1 score of 0.635 by around 5%, obtained in 6.9.

In this experiment, we had expected for better results, since, we used task-specific Hindi embeddings instead of general Hindi embeddings.

Experiment IV.B

In this experiment, we use the model architecture described in 6.1.2 with Hindi task-specific embeddings as learnt using non-linear mapping in 6.1.3.

Epochs	Precision	Recall	F1
100	0.678	0.671	0.660
150	0.696	0.686	0.677
200	0.699	0.686	0.679
250	0.710	0.699	0.692

Table 6.11: Results IV.B

Discussion and Analysis

Non linear mapping further pushes the F1 score to 0.692 from 0.686. This could be due to non-linear dependency between general and task-specific embedding spaces. Non-linear mappings capture more complex relationship between input and output spaces.

6.4.5 Experiment V : Bilingual embeddings

In this experiment, we explore the effectiveness of Bilingual embeddings. In 2.2, we talked about a method of learning bilingual embeddings which was proposed by [Mikolov et al., 2013b, Smith et al., 2017] . In this experiment, we use the transformation matrix W for mapping English-Hindi embeddings onto the same space, provided by [Smith et al., 2017] and available at ⁷ to find the nearest English words to a few Hindi words. We took five Hindi words which correspond to five distinct emotion classes : दुख, गुस्सा, डर, पर्सन्न, आश्चर्य . In English, these can be broadly translated as : **sadness, anger, fear, happy, surprise**. Word vectors of these Hindi words are first transformed to the space of English embedding space by multiplying the vectors with the transformation matrix W . Now these five vectors share the embedding space with English word vectors. 'Nearness' is measured using **Cosine distance**($cost - dist$)metric. Consider the set of English words mapped on this space is S . An English word $w_e \in S$ is the nearest word to a Hindi word w_h in the shared embedding space, if, $cos - dist(v_{w_e}, v_{w_h})$ is the least among $cos - dist(v_{w_{e'}}, v_{w_h})$, $\forall w_{e'} \in S$. Here, v_{w_h} denotes the word vector of w_h , $v_{w_{e'}}$ denotes the word vector of words $w_{e'}$ and v_{w_e} is the word vector of w_e in the shared embedding

⁷https://github.com/babylonhealth/fastText_multilingual

space.

Hindi word	5 nearest English words
दुख	unhappiness, sorrow, sadness, regret, despair
गुस्सा	disgusted, anger, commotion, infuriation
डर	fear, anger, scared, panic, frightened
पर्सन्न	pleased, bounteous, gratified, appeased, delighted
आश्चर्य	wonder, astonishment, amazed, flabbergast, astonished

Table 6.12: Bilingual embedding space

From the above table, we can see that similar meaning words of both the languages are mapped nearby in the shared bilingual embedding space. This shows that bilingual word embeddings capture semantic context of words in both languages well.

Chapter 7

Conclusion & Future Scope

7.1 Conclusion

This work explored Emotion mining in Hindi by applying cross-lingual transfer approach and learning cross-lingual task-specific embedding for Hindi using linear and non-linear methods.

Hindi is a low resource language and does not have sufficient resources to develop good quality emotion detecting system. We used a cross-lingual approach to develop Emotion detecting system in Hindi using knowledge from resource rich language English, which has a large number of annotated datasets and resources. We first trained our LSTM model with just the Hindi dataset(baseline model), which still, had a lot of room for improvement. Then, by pre-training the model with English emotion dataset and using cross-lingual embeddings in the embedding layer(cross-lingual transfer model), and then, training the model with Hindi dataset, we were able to push F1 score further and improve the model. Then, we learnt linear and non-linear mappings to from English monolingual embedding space to English task-specific embedding space to ultimately learn Hindi task-specific embeddings from bilingual Hindi embeddings. Using these task-specific embeddings further improved the F1 score to 0.692, which is the best that we got.

7.2 Future Work

We leave below some open research ideas for researchers who would like to work in this area.

- New methods of cross-lingual transfer learning can be investigated.
- Better approaches to training task-specific embeddings for low resource languages can be explored.
- Since, for low resource languages like Hindi, there is a scarcity of annotated resources, which also require manual intervention, other methods like distant supervision can be employed to annotate large datasets.
- Unsupervised approaches to Emotion mining can also be explored by creating lexicons, for which, ideas can be borrowed from works already done in English language.

Bibliography

- [Agrawal et al., 2018] Agrawal, A., An, A., and Papagelis, M. (2018). Learning emotion-enriched word representations. In *Proceedings of the 27th International Conference on Computational Linguistics*. Association for Computational Linguistics.
- [Ahmad et al., 2020] Ahmad, Z., Jindal, R., Ekbal, A., and Bhattacharyya, P. (2020). Borrow from rich cousin: transfer learning for emotion detection using cross lingual embedding. *Expert Systems with Applications*.
- [Bhowmick et al., 2009] Bhowmick, P. K., Basu, A., and Mitra, P. (2009). Reader perspective emotion analysis in text through ensemble based multi-label classification framework. *Computer and Information Science*, 2(4):64–74.
- [Bojanowski et al., 2017] Bojanowski, P., Grave, E., Joulin, A., and Mikolov, T. (2017). Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics*, 5:135–146.
- [Chaumartin, 2007] Chaumartin, F.-R. (2007). Upar7: A knowledge-based system for headline sentiment tagging. In *Proceedings of the 4th international workshop on semantic evaluations*, pages 422–425. Association for Computational Linguistics.
- [Collobert et al., 2011] Collobert, R., Weston, J., Bottou, L., Karlen, M., Kavukcuoglu, K., and Kuksa, P. (2011). Natural language processing (almost) from scratch. *Journal of machine learning research*, 12(Aug):2493–2537.
- [De Choudhury et al., 2013] De Choudhury, M., Gamon, M., Counts, S., and Horvitz, E. (2013). Predicting depression via social media. In *Seventh international AAAI conference on weblogs and social media*.

- [Deisenroth et al., 2020] Deisenroth, M. P., Faisal, A. A., and Ong, C. S. (2020). *Mathematics for machine learning*. Cambridge University Press.
- [Ekman, 1992] Ekman, P. (1992). An argument for basic emotions. *Cognition & emotion*, 6(3-4):169–200.
- [Fox, 2008] Fox, E. (2008). *Emotion Science: Cognitive and Neuroscientific Approaches to Understanding Human Emotions*.
- [Goldberg, 2017] Goldberg, Y. (2017). Neural network methods for natural language processing. *Synthesis Lectures on Human Language Technologies*, 10(1):1–309.
- [Goodfellow et al., 2016] Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep learning*. MIT press.
- [Grave et al., 2018] Grave, E., Bojanowski, P., Gupta, P., Joulin, A., and Mikolov, T. (2018). Learning word vectors for 157 languages. In *Proceedings of the International Conference on Language Resources and Evaluation (LREC 2018)*.
- [Gupta et al., 2013] Gupta, N., Gilbert, M., and Fabbriozio, G. D. (2013). Emotion detection in email customer care. *Computational Intelligence*, 29(3):489–505.
- [Hochreiter and Schmidhuber, 1997] Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8):1735–1780.
- [Jain et al., 1996] Jain, A. K., Mao, J., and Mohiuddin, K. M. (1996). Artificial neural networks: A tutorial. *Computer*, 29(3):31–44.
- [Kim et al., 2010] Kim, S. M., Valitutti, A., and Calvo, R. A. (2010). Evaluation of unsupervised emotion models to textual affect recognition. In *Proceedings of the NAACL HLT 2010 Workshop on Computational Approaches to Analysis and Generation of Emotion in Text*, pages 62–70. Association for Computational Linguistics.
- [Luyckx et al., 2012] Luyckx, K., Vaassen, F., Peersman, C., and Daelemans, W. (2012). Fine-grained emotion detection in suicide notes: A thresholding approach to multi-label classification. *Biomedical informatics insights*, 5:BII–S8966.

- [Mikolov et al., 2013a] Mikolov, T., Chen, K., Corrado, G., and Dean, J. (2013a). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.
- [Mikolov et al., 2013b] Mikolov, T., Le, Q. V., and Sutskever, I. (2013b). Exploiting similarities among languages for machine translation. *arXiv preprint arXiv:1309.4168*.
- [Mikolov et al., 2013c] Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., and Dean, J. (2013c). Distributed representations of words and phrases and their compositionality. In *Advances in neural information processing systems*, pages 3111–3119.
- [Mohammad et al., 2018] Mohammad, S., Bravo-Marquez, F., Salameh, M., and Kiritchenko, S. (2018). Semeval-2018 task 1: Affect in tweets. In *Proceedings of the 12th international workshop on semantic evaluation*, pages 1–17.
- [Mohammad and Turney, 2010a] Mohammad, S. and Turney, P. (2010a). Emotions evoked by common words and phrases: Using mechanical turk to create an emotion lexicon. In *Proceedings of the NAACL HLT 2010 Workshop on Computational Approaches to Analysis and Generation of Emotion in Text*. Association for Computational Linguistics.
- [Mohammad, 2012] Mohammad, S. M. (2012). # emotional tweets. In *Proceedings of the First Joint Conference on Lexical and Computational Semantics- Volume 1: Proceedings of the main conference and the shared task, and Volume 2: Proceedings of the Sixth International Workshop on Semantic Evaluation*, pages 246–255. Association for Computational Linguistics.
- [Mohammad and Turney, 2010b] Mohammad, S. M. and Turney, P. D. (2010b). Emotions evoked by common words and phrases: Using mechanical turk to create an emotion lexicon. In *Proceedings of the NAACL HLT 2010 workshop on computational approaches to analysis and generation of emotion in text*, pages 26–34. Association for Computational Linguistics.
- [Neviarouskaya et al., 2007] Neviarouskaya, A., Prendinger, H., and Ishizuka, M. (2007). Textual affect sensing for sociable and expressive online communication.

In *International Conference on Affective Computing and Intelligent Interaction*, pages 218–229. Springer.

- [Neviarouskaya et al., 2009] Neviarouskaya, A., Prendinger, H., and Ishizuka, M. (2009). Compositionality principle in recognition of fine-grained emotions from text. In *Third International AAAI Conference on Weblogs and Social Media*.
- [Pennebaker et al., 2001] Pennebaker, J. W., Francis, M. E., and Booth, R. J. (2001). Linguistic inquiry and word count: Liwc 2001. *Mahway: Lawrence Erlbaum Associates*, 71(2001):2001.
- [Pennington et al., 2014] Pennington, J., Socher, R., and Manning, C. (2014). Glove: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics.
- [Plutchik, 1984] Plutchik, R. (1984). Emotions: A general psychoevolutionary theory. *Approaches to emotion*, 1984:197–219.
- [Rangel and Rosso, 2016] Rangel, F. and Rosso, P. (2016). On the impact of emotions on author profiling. *Information processing & management*, 52(1):73–92.
- [Sakuma and Yoshinaga, 2019] Sakuma, J. and Yoshinaga, N. (2019). Multilingual model using cross-task embedding projection. In *Proceedings of the 23rd Conference on Computational Natural Language Learning (CoNLL)*, pages 22–32.
- [Smith et al., 2017] Smith, S. L., Turban, D. H. P., Hamblin, S., and Hammerla, N. Y. (2017). Offline bilingual word vectors, orthogonal transformations and the inverted softmax. *CoRR*, abs/1702.03859.
- [Strapparava and Valitutti, 2004] Strapparava, C. and Valitutti, A. (2004). WordNet affect: an affective extension of WordNet. In *Proceedings of the Fourth International Conference on Language Resources and Evaluation (LREC’04)*. European Language Resources Association (ELRA).
- [Strapparava et al., 2004] Strapparava, C., Valitutti, A., et al. (2004). Wordnet affect: an affective extension of wordnet. In *Lrec*, volume 4, page 40. Citeseer.

- [Tang et al., 2014] Tang, D., Wei, F., Yang, N., Zhou, M., Liu, T., and Qin, B. (2014). Learning sentiment-specific word embedding for twitter sentiment classification. In *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics.
- [Voeffray, 2011] Voeffray, S. (2011). Emotion-sensitive human-computer interaction (hci): State of the art-seminar paper. *Emotion Recognition*, pages 1–4.
- [Yadollahi et al., 2017] Yadollahi, A., Shahraki, A. G., and Zaiane, O. R. (2017). Current state of text sentiment analysis from opinion to emotion mining. *ACM Comput. Surv.*

Publications

1. "Cross-lingual transfer for Emotion mining in Hindi." Garima Jain, Kavi Narayana Murthy. *to be submitted for publication*

"Fine-tuning bilingual embeddings for Emotion mining in Hindi"

by Garima Jain

Submission date: 23-Jun-2020 12:09PM (UTC+0530)

Submission ID: 1348451883

File name: thesis-for-plag-check.pdf (232.03K)

Word count: 8682

Character count: 43217

"Fine-tuning bilingual embeddings for Emotion mining in Hindi"

ORIGINALITY REPORT

8%

SIMILARITY INDEX

3%

INTERNET SOURCES

6%

PUBLICATIONS

7%

STUDENT PAPERS

PRIMARY SOURCES

1

Submitted to University of Glasgow

Student Paper

<1%

2

Submitted to University of Bucharest

Student Paper

<1%

3

www.coursehero.com

Internet Source

<1%

4

Piotr Bojanowski, Edouard Grave, Armand Joulin, Tomas Mikolov. "Enriching Word Vectors with Subword Information", Transactions of the Association for Computational Linguistics, 2017

Publication

<1%

5

Jie, He, Yang Yuexiang, Qiao Yong, and Tang Chuan. "Accurate classification of P2P traffic by clustering flows", China Communications, 2013.

Publication

<1%

6

corpora.linguistik.uni-erlangen.de

Internet Source

<1%

7

Submitted to University of Southampton

Student Paper

<1%